



IPD-434 Seminario de Soft Computing



Introducción a Soft Computing

Dr. Nicolás Gálvez Ramírez
Dr. Patricio Olivares Roncagliolo

Departamento de Electrónica
Universidad Técnica Federico Santa María

Soft Computing

- El término **Soft Computing** fue introducido por Lotfi A. Zadeh.
- Soft Computing es un conjunto de técnicas capaces de resolver **problemas de alta complejidad** con técnicas **imprecisas o incompletas**.

Ejemplos

Estacionar automóvil en un espacio estrecho

No se necesitan las dimensiones **exactas** del espacio o del automóvil.

Reconocer caracteres escritos a mano

Aunque las letras escritas a mano no sigan su forma ideal, igualmente **somos capaces de reconocerlos**.

Hormigas recolectando comida

Inicialmente las hormigas buscan comida aleatoriamente. Cuando una de ellas encuentra comida, **comparte el mensaje** con el resto para que todas se dirijan a la fuente de alimento.

Componentes de Soft Computing

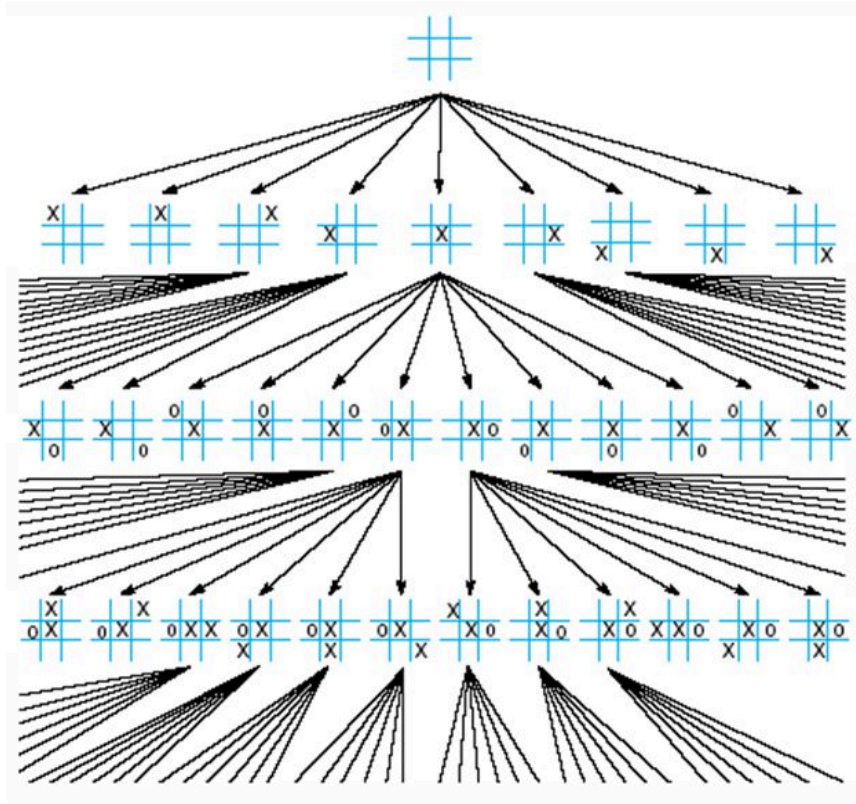
- Sistemas difusos.
- Estrategias en espacio de búsqueda.
- Computación evolutiva y bioinspirada.
- Redes Neuronales Artificiales.
- Sistemas híbridos.

Conceptos importantes

Espacio (de Estados) de Búsqueda

- Corresponde al conjunto, discreto, de estados o configuraciones posibles para un sistema.
 - Cantidad de soluciones (factibles e infactibles) de un problema.
 - Cantidad de posibles recomendaciones que una ANN puede entregar.
- Define la dificultad de un problema.
 - Conjunto finito de tamaño constante.
 - Conjunto finito de gran tamaño.
 - Conjunto finito creciente por instancia.
- Búsqueda: desde un **estado inicial** hacia un **estado final**.
 - Este estado final se define en base al problema a abordar o resolver.
 - Transformaciones que llevan desde un estado inicial a uno final.

Ejemplo



Tipos de Búsqueda

- Búsqueda Completa: Busca el mejor elemento global en todo el espacio (de estados) de búsqueda.
 - Hard Computing.
- Búsqueda Incompleta: Busca mejor elemento local en una aproximación del espacio (de estados) de búsqueda.
 - **Soft Computing**

Complejidad Computacional de Problemas

Problema de Decisión

- Problema con resultado binario: ¿Sí o No?
 - ¿Se puede decidir si un número n es primo?
 - ¿Se puede decidir si los primeros n números naturales son primos?
- Buscamos: Clases de problemas.

Clase P

- Conjunto de problemas de decisión, que pueden ser resueltos por una **Maquina de Turing determinista** en tiempo polinomial.
 - Informalmente: Problemas que pueden ser resueltos (decididos) por un algoritmo determinista en tiempo polinomial.
 - Determinista: algoritmo que sigue siempre pasos conocidos.
 - Un algoritmo resuelve diferentes instancias de distintos problemas tipo P.
 - $P = \text{Polynomial}$.

Clase NP

- Conjunto de problemas de decisión, que pueden ser resueltos por una **Maquina de Turing no determinista** en tiempo polinomial.
 - Informalmente: Problemas decidibles (sí, por respuesta) cuyas soluciones pueden ser verificadas en tiempo polinomial.
 - Resolución: Difícil (a menos que también esté en P).
 - No Determinista: algoritmo que no sigue siempre pasos conocidos.
 - Un algoritmo no resolverá de igual forma diferentes instancias de distintos problemas tipo NP.
 - $NP = \text{Non-deterministic Polynomial}$.

NP-Complete

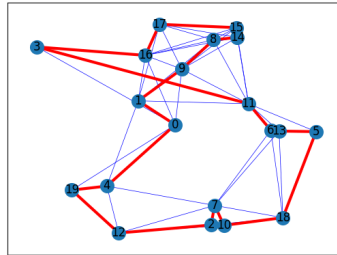
- **NP-Complete** son aquellos problemas de tipo *NP* tales que **cualquier otro** problema NP puede ser reducido a este en tiempo polinomial.
 - Reducción: función de transformación.
- **Siguen siendo NP**: Difícil de resolver, fácil de verificar solución.
- Papers base:
 - Todo SAT es NP-Completo. [1] Stephen Cook, The complexity of theorem-proving procedures. 1971. <https://dl.acm.org/doi/10.1145/800157.805047>
 - 21 Problemas NP-Completo [2] Richard Karp, Reducibility Among Combinatorial Problems, 1972. <https://cgi.di.uoa.gr/~sgk/teaching/grad/handouts/karp.pdf>

NP-Hard

- Se les llama problemas **NP-Hard** a aquellos que, si bien **no necesariamente** se encuentran en el conjunto NP, pueden llegar a ser **tan difíciles** como los

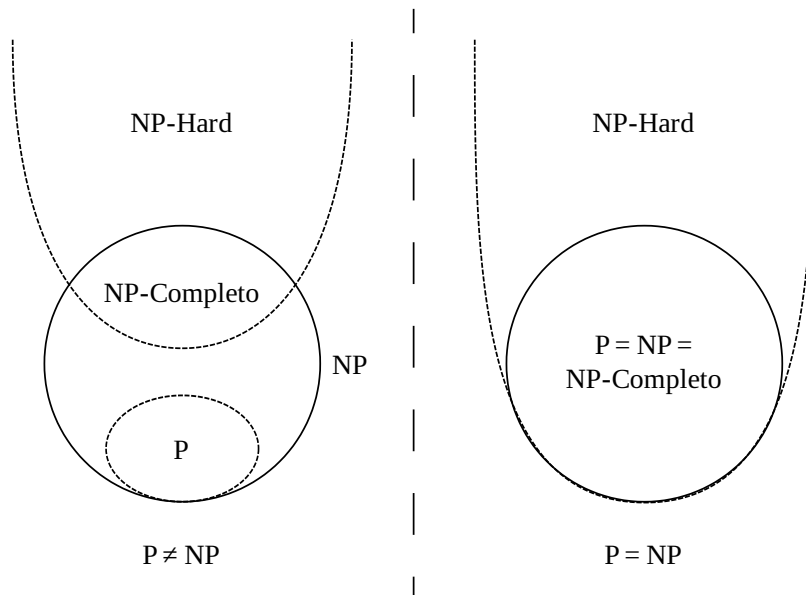
problemas más complejos de NP.

- Formalmente: Un problema es **NP-Hard** si un problema **NP-Completo** puede reducir a él.
 - Se consideran **Máquinas de Turing no deterministas** para decidir (resolver) y verificar la solución al problema en tiempo polinomial.
- Un problema NP-Hard, puede incluso no ser un problema de decisión.
- Informalmente: Problemas cuya resolución y verificación son difíciles.
- Ejemplo más famoso: Travelling Salesman Problem (TSP).



¿P = NP?

- Problema esencial de la complejidad teórica.
- NP-Complete, NP-Hard: Importancia radical para resolver este problema.
 - **Si hubiese** una Máquina de Turing Determinista que resolviera en tiempo polinomial un *problema de decisión* NP-Hard, resolveremos todos los problemas NP-Complete.
 - **Si hubiese** una Máquina de Turing Determinista que resolviera en tiempo polinomial un problema NP-Complete, resolveremos todos los problemas tipo NP, incluyendo los de tipo P.
- **Santo Grial**: Un algoritmo para resolver todo.



- [3] Richard E. Ladner, On the Structure of Polynomial Time Reducibility, 1975.
<https://dl.acm.org/doi/10.1145/321864.321877>